

System Requirements Specification (SRS)

Project: Cebuana Lhuillier Parking Reservation System
Version: 2.1
Date: March 5, 2026
Related: [Business Requirements Document](#)

1. Purpose

This document maps each business requirement (user story) from the [BRD](#) to its system-level implementation, detailing the API endpoints, database operations, UI components, and validation logic.

2. Functional Requirements Traceability

2.1 Authentication & Account Management

User Story	System Implementation
US-AUTH-01 Login	API: <code>POST /api/auth/login</code> — Validates email/password with bcrypt. Checks <code>is_blocked</code> flag — returns 403 <code>"Your account has been blocked. Please contact your administrator."</code> if true. Creates JWT token (30min TTL), issues HttpOnly secure cookie, creates DB session record with device fingerprint. UI: <code>/login</code> page with frontend validation for empty fields (toast error shown before API call); role-based redirect (user → <code>/dashboard</code> , admin → <code>/admin</code> , attendant → <code>/attendant</code>). DB: <code>users.password</code> (bcrypt hash), <code>sessions</code> collection.
US-AUTH-02 First-login password change	API: <code>POST /api/auth/change-password</code> — Validates current password, hashes new one, clears <code>must_change_password</code> flag. UI: <code>ChangePassword</code> dialog triggered by <code>must_change_password=true</code> in auth context. DB: <code>users.must_change_password</code> boolean field.
US-AUTH-03 Session timeout (30min)	Config: <code>JWT_EXPIRATION_HOURS=0.5</code> in <code>.env</code> . Implementation: JWT <code>exp</code> claim set to <code>now + 30min</code> . Cookie <code>max-age=1800</code> . Backend validates <code>exp</code> on every authenticated request via <code>get_current_user</code> dependency. Returns 401 on expiry.

User Story	System Implementation
US-AUTH-04 Logout	API: <code>POST /api/auth/logout</code> — Revokes session in DB, clears auth cookie. UI: Logout button in sidebar/header calls API and redirects to <code>/login</code> .
US-AUTH-05 Admin password reset	API: <code>PUT /api/users/{user_id}/reset-password</code> — Admin-only. Hashes new password, sets <code>must_change_password=true</code> . UI: Reset Password dialog in User Management actions dropdown.
US-AUTH-06 Self-service password change	API: <code>POST /api/auth/change-password</code> — Authenticated user submits <code>current_password</code> + <code>new_password</code> . Backend verifies current password hash (bcrypt), rejects with 400 if incorrect. Also rejects with 400 if <code>new_password</code> matches the current password hash (same-password prevention). Hashes and saves new password. UI: "Change Password" section on <code>/profile</code> page with current/new/confirm fields. DB: <code>users.password</code> updated.
US-AUTH-07 Forgot password	API: <code>POST /api/auth/forgot-password</code> — Accepts <code>email</code> . Looks up user; if found, creates an in-app notification for all admin users. Always returns 200 regardless of whether the email exists (prevents email enumeration). UI: "Forgot Password?" link on login page opens <code>ForgotPassword.jsx</code> form. DB: <code>notifications</code> collection (admin notification created).

2.2 Vehicle Management

User Story	System Implementation
US-VEH-01 Register vehicle	API: <code>POST /api/vehicles</code> — Creates vehicle with <code>plate_number</code> , <code>make</code> , <code>model</code> , <code>color</code> , <code>building_id</code> . Auto-links to authenticated user via <code>user_id</code> . Validation: Plate number normalized to uppercase, checked for uniqueness across <code>vehicles</code> collection. Length validation: Minimum 2 characters, maximum 15 characters (Pydantic validators). Duplicate plates rejected with 400 error. DB: <code>vehicles</code> collection.
US-VEH-02 Manage multiple vehicles	API: <code>GET /api/vehicles</code> (list), <code>DELETE /api/vehicles/{id}</code> (remove). Filtered by authenticated user's ID. UI: <code>/vehicles</code> page with vehicle cards, add/delete functionality.
US-VEH-03 Dashboard vehicle display	UI: <code>UserDashboard.js</code> "My Vehicles" section shows first vehicle card with "Manage >" link.

2.3 Parking Reservation

User Story	System Implementation
US-RES-01 Book parking slot	API: <code>POST /api/reservations</code> — Accepts <code>building_id</code> , <code>floor_id</code> , <code>slot_id</code> , <code>vehicle_id</code> , <code>dates[]</code> , <code>start_time</code> , <code>end_time</code> , <code>booking_type</code> , <code>reason</code> . Creates one reservation per date. Validates slot availability atomically (checks <code>slot_id+date+status</code> compound index). Generates QR token. Policy enforcement: For buildings with a <code>requires_sticker</code> policy, validates that the selected vehicle's plate has an active <code>slot_registration</code> in the building. For non-open floors, the vehicle must be registered to the specific slot being booked. UI: <code>/book</code> page with building → floor → date → slot → vehicle flow. DB: <code>reservations</code> collection with compound index on <code>(slot_id, date, status)</code> .
US-RES-02 Multi-date booking	API: <code>POST /api/reservations</code> accepts <code>dates: List[str]</code> . Backend loops over each date, creates individual reservation documents. UI: Calendar picker allows selecting multiple discrete dates (not range).
US-RES-03 View available slots	API: <code>GET /api/slots/available?floor_id=X&date=Y</code> — Returns slots not booked for that date (excludes maintenance status). UI: Slot grid on booking page, color-coded (available=green, taken=red, maintenance=gray).
US-RES-04 QR code	API: <code>GET /api/reservations/{id}/qr</code> — Returns QR code image (base64 PNG). Generated using <code>qrcode</code> library with reservation <code>qr_token</code> . API: <code>GET /api/scan/{qr_token}</code> — Returns reservation details (attendant/admin only).
US-RES-05 Cancel reservation	API: <code>PUT /api/reservations/{id}/cancel</code> — Sets status to <code>cancelled</code> . Sends notification to user. UI: Cancel (X) button on active reservation cards.
US-RES-06 Active/History tabs	UI: <code>UserDashboard.js</code> Reservations section with "Active" (today + future, non-cancelled) and "History" (past + cancelled) tabs. Backend returns all user reservations; filtering is client-side.
US-RES-07 Default booking times	Priority: User preference → Admin per-user setting → Building config → System default (08:00-18:00). API: <code>GET /api/auth/me</code> returns <code>default_start_time</code> / <code>default_end_time</code> . Building config from <code>GET /api/parking-config/{building_id}</code> .
US-RES-08 Profile booking preferences	API: <code>PUT /api/auth/booking-preferences</code> — Updates <code>default_start_time</code> / <code>default_end_time</code> on user record. UI: <code>/profile</code> page with time inputs and Save/Reset buttons.
US-RES-09 Zone building count	UI: <code>UserDashboard.js</code> "Buildings" stat card. Calls <code>GET /api/zones/user-buildings</code> to get zone-filtered building IDs, displays count. Falls back to total if no zone.

User Story	System Implementation
US-RES-10 Notifications	API: <code>GET /api/notifications</code> (list), <code>GET /api/notifications/unread-count</code> , <code>PUT /api/notifications/{id}/read</code> , <code>PUT /api/notifications/read-all</code> . UI: <code>NotificationBell</code> component in header with badge count and dropdown panel. Triggers: Booking confirmation, cancellation, no-show notification, waitlist slot available.
US-RES-11 Hourly slot timeline	API: <code>GET /api/slots/timeline?building_id=X&floor_id=Y&date=Z</code> — Returns per-slot hourly availability (6AM–10PM). Each hour marked as booked or available based on active reservations. UI: Interactive hourly timeline grid on <code>BookingPage.js</code> , color-coded cells, shows after slot selection.
US-RES-12 Waitlist	API: <code>POST /api/waitlist/join</code> — Joins user to waitlist for building+date. <code>GET /api/waitlist/status</code> — Check if user is on waitlist. <code>GET /api/waitlist/count</code> — Count of waiting users. <code>DELETE /api/waitlist/{entry_id}</code> — Leave waitlist. Background: <code>check_waitlist_expiry()</code> runs every 60 seconds, expires notified entries past the configured window, and notifies the next user. DB: <code>waitlist_entries</code> collection with position-ordered queue.

2.4 Zone & Building Rules

User Story	System Implementation
US-ZONE-01 Main building booking	API: <code>POST /api/reservations</code> — If <code>booking_type=main</code> , no additional validation. User's <code>main_building</code> compared to <code>building_id</code> .
US-ZONE-02 Zone building booking	API: <code>GET /api/zones/user-buildings</code> — Returns all building IDs in user's assigned zones. Frontend filters visible buildings to this list.
US-ZONE-03 External building (within zone)	API: <code>POST /api/reservations</code> — If <code>booking_type=external</code> : enforces single date only, requires <code>reason</code> field. Backend validates building is in user's zone but not their main building.

2.5 Admin — User Management

User Story	System Implementation
US-ADM-01 Create user	API: <code>POST /api/users</code> — Admin-only. Creates user with bcrypt password, all profile fields, <code>must_change_password=true</code> . DB: <code>users</code> collection with unique <code>email</code> index.
US-ADM-02 Bulk upload	API: <code>POST /api/users/bulk-upload</code> — Accepts CSV/Excel file + default password. Parses <code>email</code> , <code>first_name</code> , <code>last_name</code> , <code>company</code> , <code>role</code> , <code>main_building</code> (resolved by name), <code>zone</code> (resolved by name). Batch email check (\$in query), batch zone assignment (\$addToSet). Returns <code>created</code> ,

User Story	System Implementation
	<code>skipped</code> , <code>zone_assignments</code> , <code>errors</code> . Template: <code>GET /api/templates/users</code> .
US-ADM-03 Block/unblock	API: <code>PUT /api/users/{id}/block</code> , <code>PUT /api/users/{id}/unblock</code> — Toggles <code>is_blocked</code> . Blocked users rejected at login.
US-ADM-04 Main building assignment	API: <code>PUT /api/users/{id}/main-building</code> — Sets <code>main_building</code> field. Also available via bulk: <code>POST /api/users/bulk-set-building</code> .
US-ADM-05 VIP/Group Head tags	API: <code>PUT /api/users/{id}/tags</code> — Sets <code>tags[]</code> array (e.g., <code>["vip", "group_head"]</code>). Displayed as badges in user table.
US-ADM-06 Default booking times per user	API: <code>PUT /api/users/{id}</code> — Admin update includes <code>default_start_time</code> / <code>default_end_time</code> . Shown in user edit dialog.
US-ADM-07 Advanced filters	UI: <code>UserManagement.js</code> — Dropdown filters: Role, Company, Zone (with "No Zone"), Main Building (with "No Building"). Text search across name/email/company. Count badge shows filtered results.
US-ADM-08 Bulk modify	API: <code>POST /api/users/bulk-set-building</code> (body: <code>user_ids[]</code> , <code>main_building</code>), <code>POST /api/users/bulk-assign-zone</code> (body: <code>user_ids[]</code> , <code>zone_id</code>). UI: Checkboxes in table, bulk action bar appears on selection with "Set Main Building" and "Assign to Zone" buttons. "Select all X filtered" link for quick batch selection.
US-ADM-09 Template download	API: <code>GET /api/templates/users</code> — Returns CSV with columns: <code>email</code> , <code>first_name</code> , <code>last_name</code> , <code>company</code> , <code>role</code> , <code>job_family</code> , <code>main_building</code> , <code>zone</code> and example rows.
US-ADM-10 Audit timestamps	DB: <code>users.created_at</code> (set on creation), <code>users.updated_at</code> (set on every update). UI: "Created" column in table, "edited MM/DD" sub-text when updated.
US-ADM-11 Waitlist configuration	API: <code>POST /api/parking-config</code> — Fields: <code>waitlist_enabled</code> (boolean), <code>waitlist_notification_window_minutes</code> (default 15). UI: Toggle and input field in Parking Config page. Background: <code>check_waitlist_expiry()</code> task uses this config.
US-ADM-12 Job Family field	API: <code>POST /api/users</code> and <code>PUT /api/users/{id}</code> — Accepts <code>job_family</code> field (e.g., "Department Head"). <code>POST /api/users/bulk-upload</code> — CSV column <code>job_family</code> supported. DB: <code>users.job_family</code> (optional string). UI: Job Family field in user create/edit form. Template CSV includes <code>job_family</code> column.

2.6 Admin — Building & Slot Management

User Story	System Implementation
US-BLD-01 Create building	API: <code>POST /api/buildings</code> — Creates with name, address. DB: <code>buildings</code> collection.
US-BLD-02 Add floors with layout	API: <code>POST /api/buildings/{id}/floors</code> (create floor), <code>POST /api/floors/{id}/layout</code> (upload image). Images stored on disk at <code>/app/backend/uploads/</code> . UI: Floor tab with upload button, image preview.
US-BLD-03 Add slots	API: <code>POST /api/floors/{floor_id}/slots</code> — Accepts <code>slots[]</code> array with <code>label</code> , <code>row</code> , <code>column</code> . Supports bulk creation. DB: <code>parking_slots</code> collection linked to <code>floor_id</code> and <code>building_id</code> .
US-BLD-04 Block/unblock slots	API: <code>PUT /api/slots/{id}/status</code> — Toggles between <code>active</code> and <code>maintenance</code> . Maintenance slots excluded from availability.
US-BLD-05 Delete slots	API: <code>DELETE /api/slots/{id}</code> — Permanent removal.
US-BLD-06 Available slots	API: <code>GET /api/slots/available?floor_id=X&date=Y</code> — Cross-references reservations to return only free, active slots.

2.7 Admin — Zone Management

User Story	System Implementation
US-ZN-01 Create zone	API: <code>POST /api/zones</code> — Creates zone with <code>name</code> , <code>building_ids[]</code> , <code>user_ids[]</code> .
US-ZN-02 Assign users	API: <code>PUT /api/zones/{id}</code> — Updates <code>user_ids[]</code> . Also via bulk: <code>POST /api/users/bulk-assign-zone</code> .
US-ZN-03 Manage buildings	API: <code>PUT /api/zones/{id}</code> — Updates <code>building_ids[]</code> .

2.8 Admin — Parking Configuration

User Story	System Implementation
US-CFG-01 Default parking hours	API: <code>POST /api/parking-config</code> — Sets <code>default_start_time</code> , <code>default_end_time</code> per building. DB: <code>parking_configs</code> collection with unique <code>building_id</code> index.

User Story	System Implementation
US-CFG-02 Booking window	API: <code>POST /api/parking-config</code> — <code>booking_window_days</code> field (default 7). Frontend enforces max selectable date.
US-CFG-03 Release time	API: <code>POST /api/parking-config</code> — <code>release_time</code> field. Slots available for booking after this time.
US-CFG-04 Auto no-show release	API: <code>POST /api/parking-config</code> — <code>no_show_release_enabled</code> + <code>no_show_release_minutes</code> . Background: <code>auto_mark_no_shows()</code> async task runs every 5 minutes. Marks expired reservations as no-show and records a <code>no_show_at</code> timestamp. Slot release is calculated as X minutes after <code>no_show_at</code> (not after reservation end time).
US-CFG-05 Main building exclusivity	API: <code>POST /api/parking-config</code> — <code>main_building_exclusive</code> (boolean, default <code>false</code>). When enabled, <code>POST /api/reservations</code> checks if the authenticated user's <code>main_building</code> matches the booking building. Admins are always exempt. UI: Toggle switch in Parking Config page labeled "Main Building Exclusive." DB: <code>parking_configs.main_building_exclusive</code> field.

2.9 Admin — Reports & Analytics

User Story	System Implementation
US-RPT-01 Dashboard stats	API: <code>GET /api/reports/stats</code> — Returns <code>total_reservations</code> , <code>confirmed</code> , <code>cancelled</code> , <code>pending</code> , <code>no_show</code> , <code>active_users</code> , <code>occupancy_rate</code> , <code>building_breakdown[]</code> .
US-RPT-02 Date/building filter	API: <code>GET /api/reports/stats?</code> <code>start_date=X&end_date=Y&building_id=Z</code> . Frontend date picker + building dropdown.
US-RPT-03 AI insights	API: <code>POST /api/reports/ai-insights</code> — Sends parking stats to Emergent LLM (GPT), returns markdown analysis. Saved to <code>ai_insights</code> collection. UI: Markdown-rendered insights with <code>react-markdown</code> + <code>remark-gfm</code> .
US-RPT-04 Insights history	API: <code>GET /api/reports/ai-insights/history</code> — Lists all saved insights with timestamps. <code>DELETE /api/reports/ai-insights/{id}</code> for cleanup.
US-RPT-05 Admin reservations	API: <code>GET /api/admin/reservations?status=X&building_id=Y&date=Z</code> — Full reservation list with user details. <code>DELETE /api/admin/reservations/{id}</code> — Admin cancels reservation; requires <code>reason</code> body field; sends in-app notification to the parker containing the reason; then triggers waitlist notification for the freed slot.

2.10 Admin — Site Customization

User Story	System Implementation
US-SITE-01 Login page editor	API: <code>GET /api/site-content</code> (public), <code>PUT /api/admin/site-content</code> (admin-only). Fields: <code>heading_line1</code> , <code>heading_highlight</code> , <code>heading_line3</code> , <code>description</code> , <code>badge1_text</code> , <code>badge2_text</code> , <code>announcement</code> . UI: Admin Settings page with live preview.

2.11 Admin — Building Policies

User Story	System Implementation
US-POL-01 Create building policy	API: <code>POST /api/building-policies</code> — Creates policy with <code>building_id</code> , <code>policy_type</code> (e.g., <code>dedicated_slot</code>), <code>enabled</code> , <code>max_users_per_slot</code> , <code>open_floor_ids</code> , <code>requires_sticker</code> . DB: <code>building_policies</code> collection.
US-POL-02 Sticker requirement	API: <code>requires_sticker</code> field on policy. During reservation creation (<code>POST /api/reservations</code>), if enabled, the selected vehicle's plate must match an active <code>slot_registration</code> in the building. Rejects with 403 if no matching vehicle registration found.
US-POL-03 Slot registration	API: <code>POST /api/slot-registrations</code> — Registers a user + vehicle to a specific slot. Fields: <code>slot_id</code> , <code>user_id</code> , <code>vehicle_plate</code> , <code>sticker_number</code> . Checks <code>max_users_per_slot</code> limit. <code>GET /api/slot-registrations</code> — Lists registrations with user names and slot labels. <code>DELETE /api/slot-registrations/{id}</code> — Revokes registration. DB: <code>slot_registrations</code> collection.
US-POL-04 Open floors	API: <code>open_floor_ids</code> array on policy. Floors in this list bypass the strict slot-assignment check, allowing any registered user to book any slot on those floors.
US-POL-05 Zone-filtered user selection	API: <code>GET /api/slot-registrations</code> + frontend filtering. The user dropdown in the policy management UI is filtered to only show users assigned to the building's zone.
US-POL-06 Vehicle dropdown	Frontend: When assigning a slot registration, the admin selects a vehicle from the user's registered vehicles (dropdown) instead of typing a plate number manually. Populated via <code>GET /api/vehicles?user_id=X</code> .

2.12 Parking Attendant

User Story	System Implementation
US-ATT-01 Daily reservations	API: <code>GET /api/attendant/daily-reservations?date=X&building_id=Y</code> — Returns reservations with user details, vehicle, slot, floor info. Attendant role required.
US-ATT-02 Confirm arrival	API: <code>PUT /api/reservations/{id}/confirm</code> — Sets status to <code>confirmed</code> . Optional: <code>POST /api/reservations/{id}/confirm-with-photo</code> for photo evidence.
US-ATT-03 Report no-show	API: <code>POST /api/attendant/reservations/{id}/report-no-show</code> — Date guard: rejects if <code>reservation.date > today</code> . Sets status to <code>no_show</code> , records <code>no_show_at</code> timestamp, increments user's <code>no_show_count</code> , creates notification. Slot release timing is based on <code>no_show_at</code> + configured release minutes. UI: "No Show" button hidden for future dates.
US-ATT-04 Today button	UI: <code>AttendantDashboard.js</code> — "Today" button appears between date display and next arrow when selected date != today. Clicking resets to current date.
US-ATT-05 QR scan	API: <code>GET /api/scan/{qr_token}</code> — Returns reservation details (attendant/admin only). UI: <code>/scan/:qrToken</code> route.
US-ATT-06 Summary cards	UI: Three cards showing count by status: Reserved (pending), Confirmed, No Shows for selected date.

2.13 Admin — Event Blocking

User Story	System Implementation
US-EVT-01 Block slots for event	API: <code>POST /api/event-blocks/</code> — Admin-only. Accepts <code>event_name</code> , <code>reason</code> , <code>building_id</code> , <code>floor_id</code> , <code>slot_ids[]</code> , <code>date</code> , <code>start_time</code> , <code>end_time</code> . Creates an <code>event_block</code> document and one <code>reservation</code> record per slot with <code>booking_type: "event_block"</code> and <code>status: "confirmed"</code> . These reservations block the slots from being booked by regular users. DB: <code>event_blocks</code> collection + <code>reservations</code> collection.
US-EVT-02 List event blocks	API: <code>GET /api/event-blocks/</code> — Returns all event blocks. Optional <code>building_id</code> filter. Returns enriched data including building name, floor label, slot labels. UI: Event Blocking admin page with a list of active blocks.
US-EVT-03 Delete event block	API: <code>DELETE /api/event-blocks/{id}</code> — Removes the event block record and all associated reservation records (frees the slots). UI: Delete button on each event block card.

3. Non-Functional Requirements Implementation

NFR	Implementation
NFR-01 Mobile responsive	TailwindCSS responsive classes (<code>sm:</code> , <code>md:</code> , <code>lg:</code>). Mobile bottom navigation on user pages. Tested at 375px width.
NFR-02 Page load < 3s	React SPA with code-splitting via <code>React.lazy</code> . API calls use <code>Promise.all</code> for parallel fetching. Auto-retry on failure.
NFR-03 API < 500ms	MongoDB indexes on all queried fields (25 indexes). Motor async driver. Connection pooling (maxPool=50).
NFR-04 200+ users	Stateless JWT auth (no session affinity needed). Horizontal scaling via container replicas. DB connection pooling.
NFR-05 Security	HttpOnly cookies, bcrypt hashing, JWT with no fallback secret, CSP/HSTS/X-Frame headers, rate limiting (5/min on login), device fingerprinting, role-based access control. Password inputs use uncontrolled components (<code>useRef</code>) to prevent plaintext credential exposure in the DOM (mitigates CWE-312). See Compliance Matrix .
NFR-06 99.9% uptime	Container orchestration (K8s/ECS), health checks, graceful shutdown, auto-restart via supervisor.
NFR-07 Browser support	Standard React 19 + ES2020 target. No browser-specific APIs used.

Previous: [BRD](#) | Next: [System Design](#)